TD/TP6: Programmation dynamique

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

06 janvier 2026

Exercice 1 : Empilement de tuiles

On souhaite construire des *tours* en empilant des tuiles de largeurs différentes.

On se donne trois paramètres :

— n, la *hauteur* d'une tour (le nombre de tuiles qui la composent)

— m, la largeur maximale possible pour une tuile (par exemple si $m = 3$, les largeurs autorisées sont 1, 2 et 3)

— k, le nombre maximal de tuiles d'une même largeur pouvant être utilisées dans une tour.

L'objectif est de déterminer, pour un triplet donné (n, m, k), le nombre de tours distinctes qu'il est possible de construire. Deux contraintes importantes doivent être respectées :

— Les tours doivent être *stables* : chaque tuile doit reposer sur une tuile de largeur supérieure ou égale à la sienne.

— Deux tours sont considérées comme différentes s'il existe au moins une hauteur $h \in \{1, \dots, n\}$ telle que les largeurs des tuiles à cette hauteur diffèrent.

Par exemple, pour $(n, m, k) = (3, 3, 2)$, il est possible de construire 7 tours différentes : [1, 1, 2], [1, 1, 3], [1, 2, 2], [1, 2, 3], [1, 3, 3], [2, 2, 3] et [2, 3, 3]. Chaque liste représente les largeurs des tuiles composant une tour, la dernière valeur correspondant à la base de la tour.

Question 1 Expliquer pourquoi l'utilisation de la programmation dynamique est adaptée à ce problème.

Question 2 Proposer une stratégie de résolution fondée sur la récursivité. Donner une relation de récurrence exprimant le nombre de tours distinctes possibles en fonction des paramètres (n, m, k), ainsi que les cas de base.

Question 3 A partir de la la relation de récurrence précédente, proposer le pseudo-code d'un algorithme récursif utilisant la mémoïsation pour résoudre le problème.

Question 4 Proposer un algorithme alternatif, sans récursivité, permettant de résoudre le même problème.

Question 5 Déterminer et justifier les complexités temporelle et spatiale de l'algorithme de la question précédente. La complexité spatiale est-elle meilleure que la version récursive ?

Exercice 2 : Problème du sac à dos

Dans cet exercice, on revient sur le problème du sac à dos, déjà abordé lors du TP sur les algorithmes gloutons. Voici un rappel de sa formalisation.

On considère un sac à dos de capacité C et $n \in \mathbb{N}$ objets différents, dont on note $p_0, \dots, p_{n-1}$ les poids et $v_0, \dots, v_{n-1}$ les valeurs. On considère que ces poids et valeurs sont *des entiers strictement positifs*, et que le poids de chaque objet ne dépasse pas la capacité totale. Le but du problème est de retourner un sous-ensemble $X \subseteq [\![0, n-1]\!]$ tel que :

— la somme des poids ne dépasse pas la capacité : $\sum_{k \in X} p_k \leqslant C$

— la somme des valeurs soit maximale : $\sum_{k \in X} v_k$ est maximale.

L'objectif de cet exercice est de déterminer **la valeur maximale** qu'il est possible d'obtenir avec une capacité C et les objets indiqués.

Question 6 Proposer une formule de récurrence pour ce problème ainsi que le ou les cas de base. La récurrence portera sur le calcul de $V(C, k)$, à savoir la valeur maximale obtenue en considérant une capacité C et les k premiers objets (indexés de 0 à $k-1$).

Question 7 Utiliser la programmation dynamique pour écrire une fonction Python `sac_a_dos_dynamique` prenant en entrée :

— `P` : liste des poids des objets,

— `V` : liste des valeurs correspondantes,

— `C` : capacité du sac.

La fonction peut être implémentée avec ou sans récursion, au choix.

Question 8 Quelle est la complexité temporelle de cette fonction ? Justifier.